

# Assignment 1

## Week 1

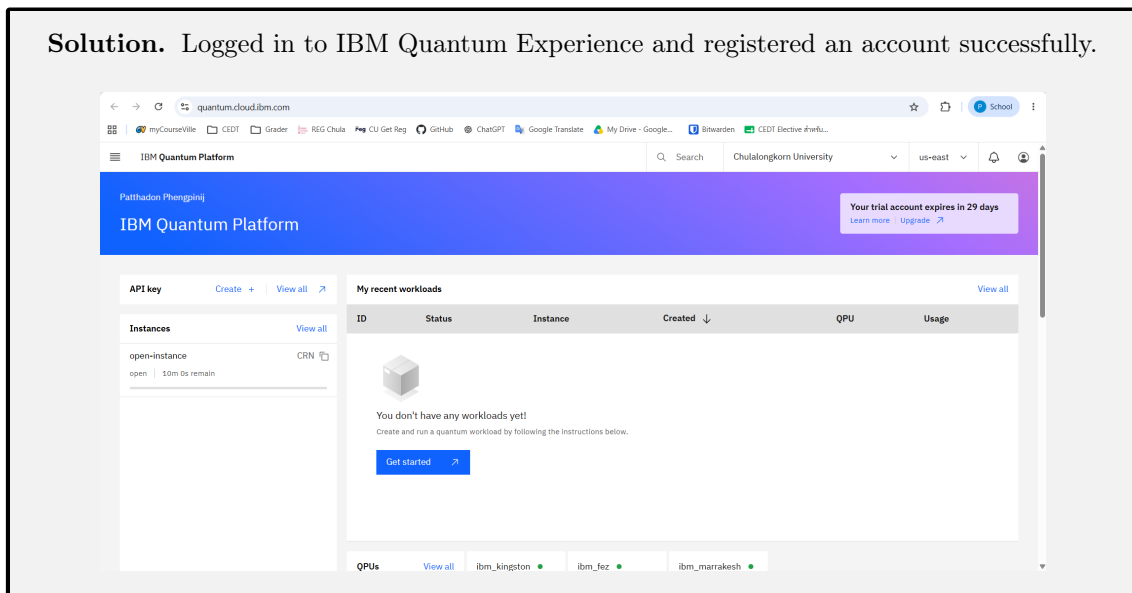
6733172621 Patthadon Phengpinij

*Collaborators.* Claude (for L<sup>A</sup>T<sub>E</sub>X styling and grammar checking)

### Problem 1. Quantum Computing Account

Please go to IBM Quantum Experience and register an account, <https://quantum.cloud.ibm.com/>.

**Solution.** Logged in to IBM Quantum Experience and registered an account successfully.



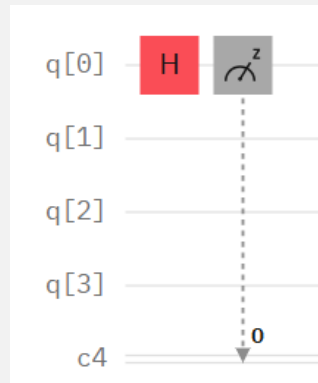
### Problem 2. My First Quantum Circuit

Try to create a circuit. Click on IBM Quantum Composer and drag the gates to form the following circuit.



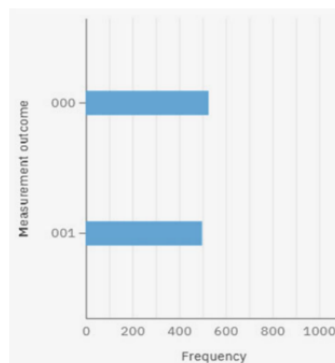
The first icon is a quantum gate called Hadamard gate and the second icon is a measurement. The input is from the left ( $q_0$ ). Do not worry if you do not understand what it is at this moment. But good job! You have created your first quantum circuit!

**Solution.** Finished creating the quantum circuit as instructed.



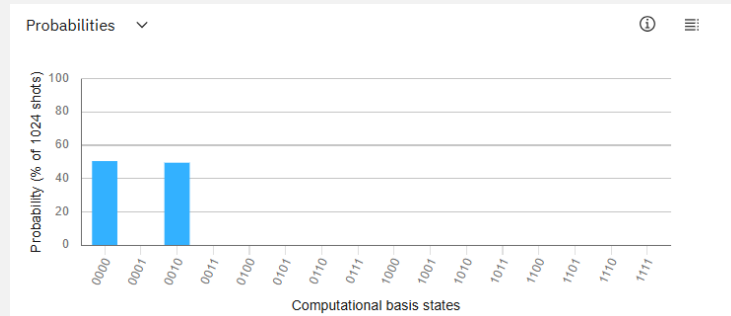
### Problem 3. My First Quantum Computing

Submit the job. Click on the result when it is finished. You see some like this. You may submit to a simulator or real quantum hardware.



We started with a state in  $|0\rangle$  (a basis state). After the *Hadamard gate*, it becomes a superposition of  $|0\rangle$  and  $|1\rangle$ , i.e.  $\sim |0\rangle + |1\rangle$  (we will learn later). Then we performed a measurement on the output and the superposition state collapses to either  $|0\rangle$  or  $|1\rangle$ . The histogram shows that about 50% of the chance we obtain  $|0\rangle$  and another 50% of the chance we obtain  $|1\rangle$ . Cool! We have done our first quantum computation. This job was submitted to a real quantum computer, called “ibmq\_casablanca.” It is a 5-qubit quantum computer. However, this computer is gone. But definitely, you will find other more powerful ones.

**Solution.** The job was done successfully and the result was obtained as instructed.



#### Problem 4. Google Colab and Python

We need to do a lot of matrix operations. We will choose Google Colab, in which we can run Python easily. Since IBM Quantum Experience also uses Python, it will save your efforts. Please open a Google Colab account here <https://colab.research.google.com/>.

**Solution.** I decided to use my local computer to run the Jupyter notebook instead of Google Colab.

#### Problem 5. Vector Inner Product Using Python

We use the NumPy library to perform vector and matrix operations. To import the library, type

```
1 import numpy as np
```

Then let's implement

$$\vec{v}_1 \cdot \vec{v}_2 = (\sqrt{3} \ 1) \begin{pmatrix} 1 \\ 1 \end{pmatrix} = (\sqrt{3} \times 1) + (1 \times 1) = \sqrt{3} + 1 \approx 2.732$$

```
1 v1 = np.array([np.sqrt(3), 1])
2 v2 = np.array([1, 1])
3 inner_product = np.vdot(v1, v2)
4
5 print(inner_product)
```

Type Shift-Enter to let it run. Do you see the same result? Note that we use `np.vdot` instead of `np.dot` because we need to use complex vector dot products in quantum computing. Try also `print(v1.shape)`, which will give you `(2, 1)`. It means we constructed a  $2 \times 1$  matrix, which is a column vector with 2 elements.

**Solution.** Try running the code in a Python environment and obtained the same result of approximately 2.732.

```
1 import numpy as np
2
3 v1 = np.array([[np.sqrt(3)], [1]])
4 v2 = np.array([[1], [1]])
5 inner_product = np.vdot(v1, v2)
6
7 print(f"Inner product: {inner_product:.3f}")
8 print("v1 Shape:", v1.shape)
9 print("v2 Shape:", v2.shape)
```

#### Problem 6. Inner Product with Complex Coefficient 1

Now, let's try  $\vec{v}_1 = \begin{pmatrix} i \\ 1 \end{pmatrix}$  and  $\vec{v}_2 = \begin{pmatrix} -i \\ 1 \end{pmatrix}$ . Firstly, try to use hand calculation. Then use Python to check if it is correct. Note that in the NumPy library, you need to put  $i$  as  $j$ . For example,  $5 + i$  should be entered as `5 + 1j` (putting the "1" is necessary).

Think about that before reading the code below:

```

1 v1 = np.array([[1j], [1]])
2 v2 = np.array([[1j], [1]])
3 inner_result = np.vdot(v1, v2)
4 print(inner_result)

```

**Solution.** First, try to calculate the inner product manually:

$$\begin{aligned}
 \vec{v}_1 \cdot \vec{v}_2 &= \begin{pmatrix} i \\ 1 \end{pmatrix} \cdot \begin{pmatrix} -i \\ 1 \end{pmatrix} \\
 &= (i^* \ 1^*) \begin{pmatrix} -i \\ 1 \end{pmatrix} \\
 &= (-i \ 1) \begin{pmatrix} -i \\ 1 \end{pmatrix} \\
 &= (-i \times -i) + (1 \times 1) \\
 &= -1 + 1 \\
 \vec{v}_1 \cdot \vec{v}_2 &= 0
 \end{aligned}$$

Then, re-check the result using Python code given by the problem. We can see that the result is also 0, which is the same as what we calculated manually.

### Problem 7. Inner Product with Complex Coefficient 2

Now, let's try  $\vec{v}_1 = \begin{pmatrix} 1+2i \\ 1 \end{pmatrix}$  and  $\vec{v}_2 = \begin{pmatrix} 1-i \\ 1 \end{pmatrix}$ .

**Solution.** First, we calculate the inner product manually:

$$\begin{aligned}
 \vec{v}_1 \cdot \vec{v}_2 &= \begin{pmatrix} 1+2i \\ 1 \end{pmatrix} \cdot \begin{pmatrix} 1-i \\ 1 \end{pmatrix} \\
 &= ((1+2i)^* \ 1^*) \begin{pmatrix} 1-i \\ 1 \end{pmatrix} \\
 &= (1-2i \ 1) \begin{pmatrix} 1-i \\ 1 \end{pmatrix} \\
 &= (1-2i)(1-i) + (1)(1) \\
 &= 1-i-2i+2i^2+1 \\
 &= 1-3i-2+1 \\
 \vec{v}_1 \cdot \vec{v}_2 &= -3i
 \end{aligned}$$

Then, re-check the result using the following Python code:

```

1 v1 = np.array([[1 + 2j], [1]])
2 v2 = np.array([[1 - 1j], [1]])
3 inner_result = np.vdot(v1, v2)
4
5 print(f"Inner product: {inner_result:.3f}")

```

We can see that the result is also -3i, which is the same as what we calculated manually.

# APPENDIX 1

Assignment 01 | Code Solution

## assignment-01-code

August 13, 2026

### 1 Assignment I

---

#### 1.1 Import External Libraries

```
[1]: import numpy as np
```

---

#### 1.2 5. Vector Inner Product Using Python

```
[2]: v1 = np.array([[np.sqrt(3)], [1]])  
v2 = np.array([[1], [1]])  
inner_product = np.vdot(v1, v2)  
  
print(f"Inner product: {inner_product:.3f}")  
print("v1 Shape:", v1.shape)  
print("v2 Shape:", v2.shape)
```

```
Inner product: 2.732  
v1 Shape: (2, 1)  
v2 Shape: (2, 1)
```

---

#### 1.3 6. Inner Product with Complex Coefficient 1

```
[3]: v1 = np.array([[1j], [1]])  
v2 = np.array([[-1j], [1]])  
inner_result = np.vdot(v1, v2)  
  
print(f"Inner product: {inner_result:.3f}")
```

```
Inner product: 0.000+0.000j
```

---

#### 1.4 7. Inner Product with Complex Coefficient 2

```
[4]: v1 = np.array([[1 + 2j], [1]])  
v2 = np.array([[1 - 1j], [1]])  
inner_result = np.vdot(v1, v2)  
  
print(f"Inner product: {inner_result:.3f}")
```

Inner product: 0.000-3.000j

---